

Laboratorio de Datos

Trabajo Práctico 02

Grupo “Sanisidrenses”



Guadalupe Cataneo

Máximo Ballestrero

Juana María Leiton

Introducción

En la actualidad se han vuelto cada vez más relevantes los modelos predictivos capaces de clasificar imágenes. Tienen aplicaciones en seguridad, medicina en el área de diagnósticos, detección de fraude, conducción autónoma y más.

Nuestro objetivo en este trabajo fue analizar distintos modelos predictivos capaces de clasificar imágenes y evaluar qué tan buenas predicciones generan, para elegir los más efectivos. Para ello trabajamos con un dataset llamado Fashion-MNIST, que posee los datos de 70.000 imágenes de distintas prendas de ropa o accesorios. Las imágenes son de 28 x 28 píxeles, en escala de grises, y cada una pertenece a una de diez clases (cada una se corresponde a algún tipo de prenda/accesorio). En las siguientes ilustraciones mostramos a modo de ejemplo tres imágenes de cada clase, y la prenda/accesorio que se corresponde a cada número de clase.

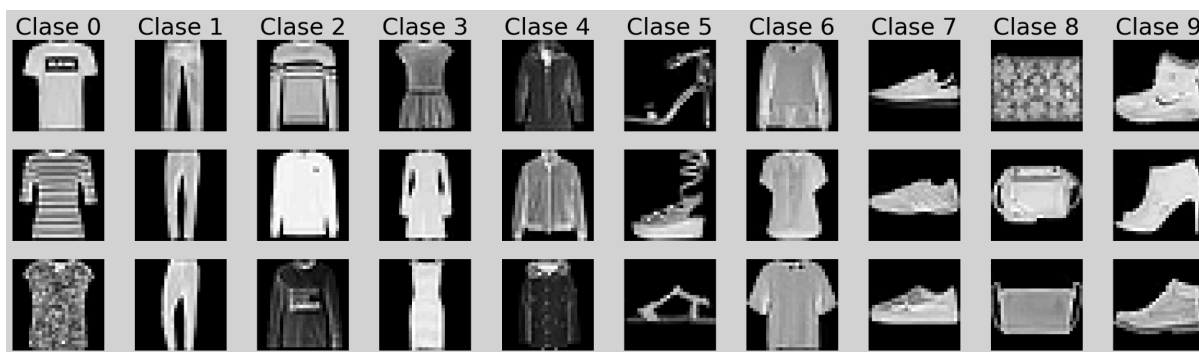


Fig 1: Imágenes ejemplo de cada clase

Clase 0	Clase 1	Clase 2	Clase 3	Clase 4	Clase 5	Clase 6	Clase 7	Clase 8	Clase 9
Remera	Pantalón	Pullover	Vestido	Abrigo	Sandalia	Camisa	Zapatilla	Bolsa	Bota

Tabla 1: Las clases y sus prendas/accesorios correspondientes

Trabajando con este dataset, ajustamos dos modelos predictivos distintos. Uno basado en el método KNN (K Nearest Neighbors) para clasificación binaria a partir de una cantidad reducida de atributos, y luego otro más general, para clasificación multiclase, basado en el método del árbol de decisión, tomando en cuenta todos los atributos. Para ambos casos fuimos ajustando los parámetros midiendo su performance a la hora de generar predicciones, con el objetivo de encontrar el modelo más efectivo

Análisis exploratorio

Comenzamos realizando un análisis exploratorio de los datos. En principio queríamos información general del dataset: la cantidad de datos, la cantidad de clases, y la cantidad y tipo de atributos. Como ya mencionamos, son 70.000 imágenes. Cada imagen tiene 786 atributos: un índice (un entero entre 0 a 69.999 para identificar cada imagen), un label (un entero entre 0 y 9 para identificar la clase), y las 784 intensidades de cada pixel (cada una un entero entre 0 y 255 que representa la luminosidad; 0 representa negro, 255

blanco). Hay 7000 imágenes de cada clase así que están distribuidas de manera balanceada.

Por otro lado, nos interesaba saber cómo se relacionaban las clases entre sí, para tratar de entender cómo funcionarían nuestros modelos predictivos. A simple vista, detectamos ciertos patrones en las imágenes de una misma clase, esto nos dio un indicio para plantearnos dos aspectos. Primero la diferencia y similitud general entre algunas clases, y luego el nivel de semejanza interna que podían llegar a tener distintas clases.

Como primer paso, se nos ocurrió calcular, para cada clase, sus imágenes promedio y mediana. Definimos la imagen promedio de una clase como la imagen que resulta de calcular la intensidad promedio de cada píxel. Análogamente, la imagen mediana de una clase es la que resulta de calcular la intensidad mediana de cada píxel. Estas dan una idea general de las imágenes de cada clase, y nos ayudarían a responder nuestras preguntas. A continuación presentamos las imágenes mencionadas para cada clase:

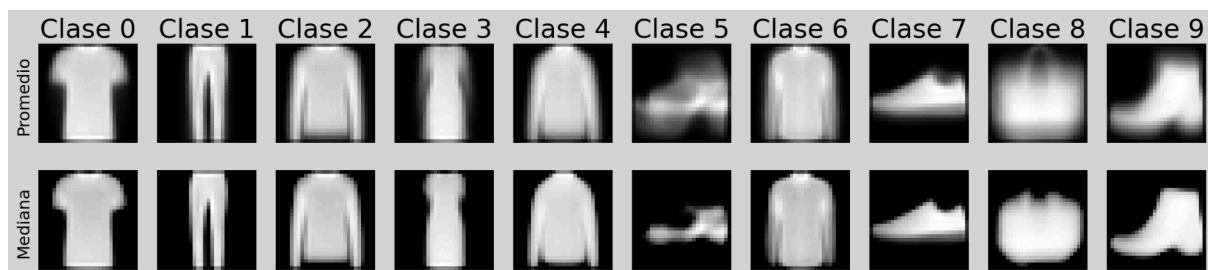


Fig. 2: Imágenes promedio y mediana de cada clase

Para ver la similitud y diferencia general entre algunas clases, tomamos a forma de ejemplo las clases 1, 2 y 6. Observamos claramente en la Fig. 2 que las clases 2 y 6 se asemejan bastante entre sí, y difieren de la clase 1. Pero para analizar más en profundidad, decidimos tomar dos medidas para estas tres clases.

Por un lado, decidimos analizar la distribución de las intensidades en las imágenes promedio de cada clase. Por otro lado, decidimos analizar la distribución de tamaños en cada clase, definiendo al tamaño de una imagen como la cantidad de píxeles de la misma cuya intensidad es distinta de cero. La idea es que como el fondo de las imágenes es siempre negro, contar la cantidad de píxeles no nulos da una noción de cuánto espacio ocupa la figura. Ambas distribuciones se pueden ver en los siguientes gráficos:

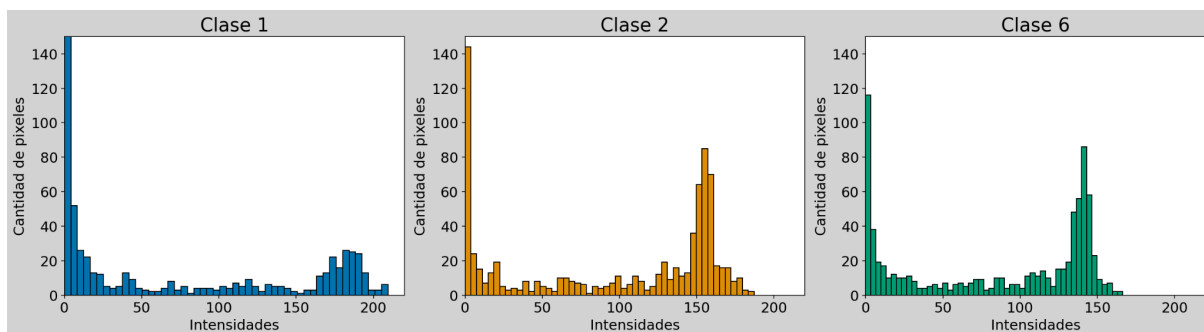


Fig. 3: Distribución de intensidad en imágenes promedios de las clases 1, 2 y 6 restringiendo la cantidad de píxeles hasta 150 (para ver con mayor claridad las distribuciones, gráfico original en el anexo, Fig. A1)

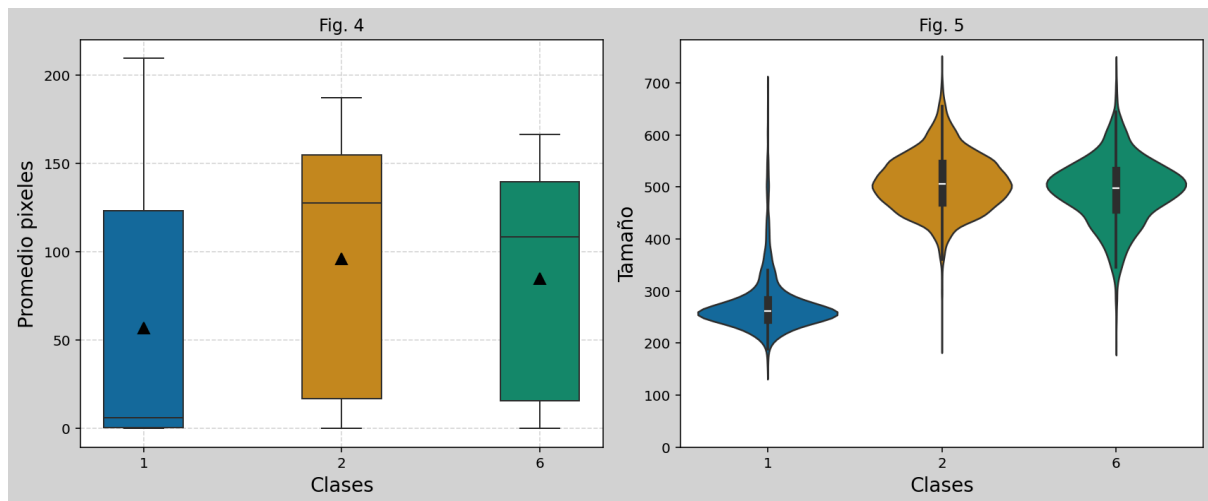


Fig 4: Boxplot de distribución de intensidad en imágenes promedios de las clases 1, 2 y 6

Fig. 5: Violín plot de la distribución de tamaño en las imágenes de las clases 1, 2 y 6

Observamos en las Fig. 3 y 4 que se asemejan bastante las distribuciones de intensidad en las imágenes promedio de las clases 2 y 6. Ambas tienen medianas bastante más altas que la clase 1. Las clases 2 y 6 parecen tener, en general, mayor concentración de píxeles con intensidad relativamente alta (más de 125), en comparación con la clase 1 que tiene una muy alta concentración de píxeles con muy baja intensidad (menos de 25). Por otro lado, observamos en la Fig. 5 que también coinciden bastante las distribuciones de tamaño entre las clases 2 y 6, y que ambas difieren de la clase 1. La clase 2 y 6 parecen tener una mayor concentración de imágenes de tamaño más grande (alrededor de 500), mientras que la mayoría de sus imágenes tienen tamaño alrededor de 300.

Concluimos en efecto, que con estos criterios se puede confirmar que las clases 2 y 6 son bastante similares entre sí, y que ambas difieren fuertemente de la clase 1. Intuimos, mirando la Fig. 2, si hiciéramos un análisis similar tomando la clase 4, cuyas imágenes promedio y mediana se asemejan bastante a las de las clases 2 y 6, llegaríamos a una conclusión parecida. Esto repercutiría en la performance de nuestros modelos multiclasa, que tendrían mayor efectividad para diferenciar entre algunas clases que otras.

En segundo lugar, para medir la similitud interna de una clase decidimos calcular el IQR de cada píxel dentro de una clase. Esto daría un indicio de que tan dispersa es la distribución de intensidad para ese píxel para esa clase, es decir, que tanto varía. Como ejemplo tomamos las clases 1 y 8, y graficamos el resultado:

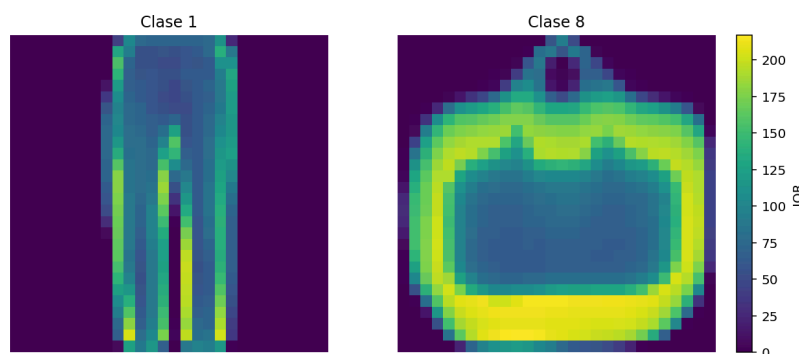


Fig. 6: IQR de cada píxel para las clases 1 y 8

Vemos en la Fig. 6 que ambas clases se mantienen relativamente constantes en sus bordes, con un IQR muy cercano a cero, pero en las regiones donde más varían, se marca la silueta de un pantalón y una cartera, respectivamente. Observamos que la clase 8 tiene muchos píxeles con IQR muy alto (cerca de 200), mientras que la clase 1 tiene pocos píxeles con nivel de IQR muy alto, la gran mayoría tienen un IQR menor a 100. Esto indica que la clase 8 tiene menos similitud interna que la 1. Concluimos entonces, que hay clases con más semejanza interna que otras.³

Para finalizar nuestro análisis exploratorio, nos interesaba saber si todos los píxeles son relevantes a la hora de clasificar. Notamos en la Fig. 2 que los bordes suelen ser siempre negros para todas las clases. Por lo tanto, intuimos que no serían relevantes para clasificar, pues parecen mantenerse constantes en todas las clases.

Para poner a prueba nuestra hipótesis analizamos las imágenes medianas presentadas en la Fig. 2: tomando estas diez imágenes, le calculamos a cada píxel su rango en la distribución de sus intensidades (es decir, cada pixel tiene una distribución de 10 valores, uno para cada imagen mediana). La idea es que si el rango para un píxel es alto, este varía bastante entre clases, por lo que lo podemos considerar más relevante para diferenciar entre ellas. Al contrario, si el rango es muy bajo, quiere decir que el píxel varía poco según clase, por lo que no servirá para diferenciar entre ellas. En la siguiente figura se muestra el rango de cada pixel:

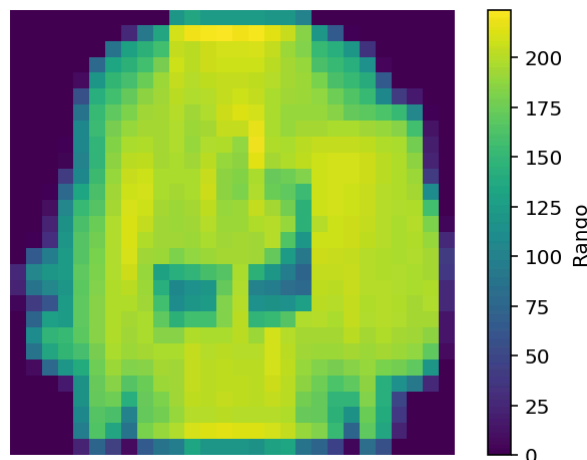


Fig. 7: Rango de píxeles a partir de la muestra de las 10 imágenes medianas de cada clase.

Observamos en la Fig. 7 que se confirma nuestra hipótesis: los píxeles ubicados en los bordes y las esquinas resultan tener menos variación a través de las clases. Es decir, parecen ser menos relevantes a la hora de clasificar.

Modelo 1 - Clasificación Binaria

En esta parte del trabajo nos interesaba comparar varios modelos de clasificación binaria basado en el algoritmo de K Nearest Neighbors (KNN), trabajando con las clases 0 y 8, tomando una cantidad reducida de atributos. En KNN se representa cada muestra como un punto en un espacio definido por sus atributos. Para clasificar una nueva muestra, se buscan los K “vecinos” más cercanos en ese espacio, y la clase asignada es la más

frecuente entre estos vecinos. En nuestro caso, las muestras son las imágenes y los atributos serían las intensidades de los distintos píxeles.

Comenzamos tomando, a partir del dataset original, un subconjunto que contiene únicamente las clases 0 y 8. Observamos en primer lugar que como ambas clases tienen 7000 imágenes, este subconjunto está balanceado. Luego, tomamos este subconjunto y lo dividimos en dos, una parte conteniendo el 80% de los datos, que llamaremos conjunto train y el otro 20% se llamará conjunto test. Vamos a entrenar los modelos con los datos del conjunto train, y luego evaluar sus predicciones en el conjunto test. Tomando en cuenta que hay la misma cantidad de imágenes para las dos clases, y que en nuestro problema nos interesa medir el porcentaje de clasificaciones correctas, decidimos comparar los modelos calculando su exactitud.

Lo primero que tuvimos que hacer fue decidir qué atributos (en nuestro caso píxeles) tomar en cuenta para ajustar a los modelos. Sabíamos que teníamos que elegir los que diferencien mejor las clases, es decir, píxeles cuyas intensidades sean en general distintas para cada clase. Para hallarlos decidimos ver en qué píxeles había mayor diferencia entre las imágenes promedio y mediana de cada clase. Para ello las restamos (esto quiere decir restar pixel a pixel el valor de las intensidades) y elegimos los píxeles con mayor valor absoluto, es decir mayor diferencia. Aquí se puede ver el resultado de restarle a las imágenes promedio y mediana de la clase 8 las correspondientes de la clase 0:

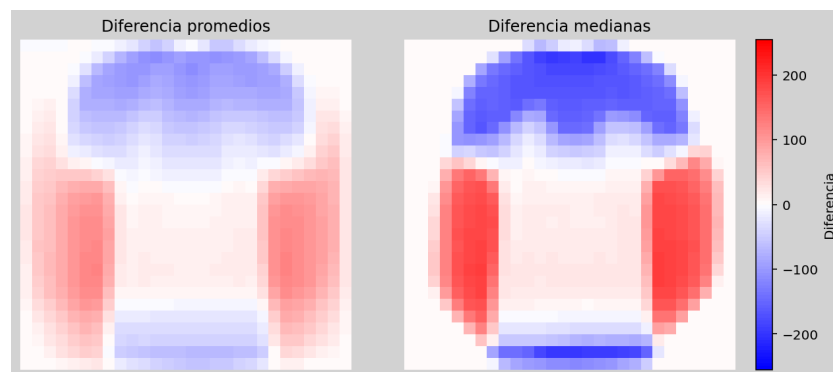


Fig. 8: La resta de las imágenes promedio y mediana entre la clase 8 y 0

Notamos en la Fig. 8 que para ambas imágenes, se marcan claramente dos regiones, una roja y una azul (si bien en la imagen de diferencia de promedios es más difuminado). Observamos que la zona roja indica la región donde suele predominar la clase 8: pues si la diferencia dio un número positivo alto, es porque la intensidad de ese pixel para la clase 8 es bastante más alta que para la clase 0. Análogamente, las zonas azules marcan donde predomina la clase 0. Uno puede confirmar esto volviendo a las imágenes promedio y mediana de cada clase en la Fig. 2.

Para este trabajo consideramos tres cantidades de atributos: 3, 5 y 10. Para cada una de estas cantidades de píxeles, elegimos tres conjuntos, basándonos en tres criterios distintos. Como primer criterio, elegimos los píxeles con mayor diferencia en las imágenes promedio (es decir tomar los píxeles con mayor valor absoluto en la imagen de la resta de imágenes promedio en la Fig. 8). Luego, como segundo criterio, tomamos los píxeles que tienen mayor diferencia en las imágenes medianas (es decir, tomar los píxeles con mayor valor absoluto en la imagen de la resta de imágenes mediana en la Fig. 8).



Tras hacer estas selecciones con estos dos criterios, notamos que los píxeles siempre caían en las mismas regiones: para el primer criterio caían en la región roja mencionada anteriormente, y para el segundo criterio caían en la zona azul (con la excepción de que cuando tomamos 10 píxeles para el segundo criterio, hay uno que si cae en la zona roja). Por lo tanto, como tercer criterio tomamos una mezcla de los píxeles obtenidos para cada criterio, escogiendo aquellos que abarcaran regiones distintas de la imagen. Puede verse ilustrado en el anexo, Fig. A11-A13, la ubicación de los píxeles seleccionados para cada criterio. En fin, ajustamos modelos a partir de 9 conjuntos de píxeles, tres criterios de selección para tres cantidades de atributos.

Para cada uno de estos conjuntos, entrenamos clasificadores de KNN con el conjunto train y calculamos su exactitud a la hora de clasificar las imágenes del conjunto test, variando el K (la cantidad de vecinos) entre los siguientes valores: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 30, 40, 50, 75, 100, 125, 150, 175 y 200. Los gráficos de exactitud en función de K para cada modelo pueden ser encontrados en el anexo, Fig. A2-A10. Consideramos que dado el tamaño del conjunto test, no sería más efectivo elegir un K mucho más grande. Los resultados se pueden ver en la siguiente tabla:

Conjunto	crit. 1 3 pix.	crit. 1 5 pix.	crit. 1 10 pix.	crit. 2 3 pix.	crit. 2 5 pix.	crit. 2 10 pix.	crit. 3 3 pix.	crit. 3 5 pix.	crit. 3 10 pix.
K de mayor exactitud	40	20	40	20	7	75	20	30	6
Mayor exactitud (%)	92.10	92.75	93.21	86.89	87.46	93.14	92.57	93.67	94.46

Tabla 2: Exactitudes máximas y los K correspondientes para cada conjunto de atributos elegido

En primer lugar, podemos ver en la Tabla 2 lo importante que es un buen criterio: consistentemente el criterio 2 obtuvo menores exactitudes que el criterio 1, y este (aunque en menor medida) obtuvo menores que el criterio 3. Aunque observamos que para los tres criterios, un aumento en la cantidad de píxeles significó un aumento en la exactitud, es curioso que, tomando 3 píxeles en base al criterio 1 o 3, se obtiene mayor exactitud que con 5 píxeles en base al criterio 2. Notamos además un aumento pronunciado en la exactitud del segundo criterio al pasar de cinco píxeles a diez, y esto es causado por lo que mencionamos previamente: al tomar diez atributos con el criterio 2, no todos caen en la misma zona. Y por último, notamos que no pudimos observar ningún patrón en los valores de K.

Finalmente, partiendo de la Tabla 2, observamos que el modelo de clasificación binaria basado en KNN con mayor exactitud que hallamos, es el que parte del conjunto de diez píxeles obtenidos a través del tercer criterio tomando la cantidad de vecinos $K=6$. En la siguiente figura se puede ver su matriz de confusión:

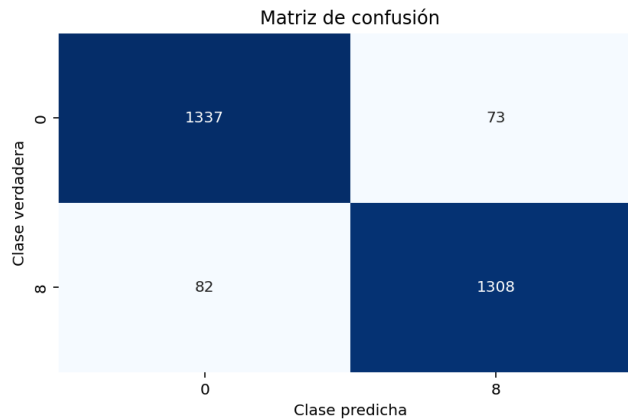


Fig 9: Matriz de confusión del mejor modelo

Como reflexión final de esta sección, reconocemos que el nivel alto de exactitud viene dado en parte por el hecho de que la clase 0 y 8 son bastante distintas (como se puede ver en la Fig. 2). En cambio, si nuestro objetivo fuese discernir entre por ejemplo las clases 2 y 6 (las cuales ya vimos que se asemejan bastante entre sí), es posible que hubiésemos alcanzado un nivel de exactitud menor.

Modelo 2 - Clasificación Multiclase

Como mencionamos anteriormente, cada imagen presente en el dataset puede pertenecer únicamente a una de diez posibles clases. Entonces, teniendo una imagen, ¿Cómo puedo saber a qué clase pertenece? En esta instancia, nos interesa ajustar un modelo capaz de predecir correctamente, en cierta medida, la clase de cualquier imagen del dataset. Decimos en cierta medida dado que al trabajar con esta cantidad de datos y posibles clases, es esperable que el modelo tenga cierto porcentaje de error a la hora de predecir, más aún conociendo la similitud presente entre ciertas clases, vista en el análisis exploratorio.

Para lograr esta propuesta, utilizamos un árbol de decisión como clasificador. Este modelo consiste en una estructura con forma de árbol, donde cada nodo representa una pregunta o condición sobre alguna característica de la imagen a predecir. Un ejemplo de una característica es la intensidad de cierto píxel elegido. Las ramas indican las posibles respuestas, y cada una deriva en un nuevo nodo, hasta alcanzar el final, donde se concreta la predicción. Este modelo tiene la peculiaridad de facilitar la visualización de la manera en la que se toman las decisiones de clasificación.

Véase un ejemplo del comienzo de un árbol de decisión acorde con los datos con los que estamos trabajando:

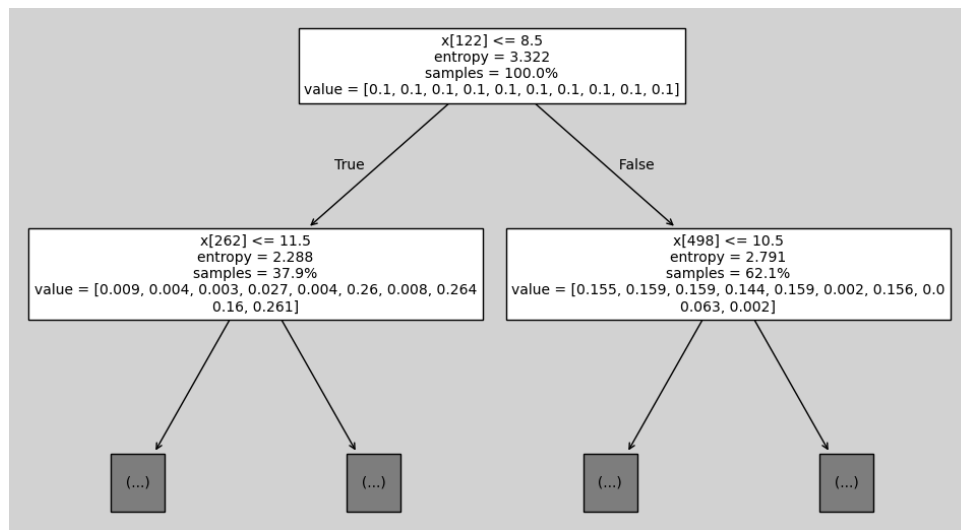


Fig. 10: Primer nivel de árbol de decisión. $x[122]$, $x[262]$ y $x[498]$ píxeles de la imagen evaluada

Comencemos con un breve estudio de los árboles de decisión. Estos pueden generarse contemplando distintas configuraciones de hiperparámetros, para así lograr ajustar el modelo lo mejor posible a cada caso. En el nuestro, nos interesan únicamente aquellos nombrados a continuación.

- **Criterion:** Mide la calidad de una división. Aquellos que usaremos son *gini* y *entropía*.
- **Max_depth:** Profundidad máxima del árbol. En nuestro caso nos interesan aquellos árboles que contemplen hasta máximo diez niveles de profundidad.
- **Random_state:** Controla la aleatoriedad del criterio del modelo. Esto permite que este mantenga sus elecciones azarosas constantes, logrando que el azar no interfiera en la elección del mejor árbol. Mantendremos este valor fijo.

Ahora que ya tenemos una noción del funcionamiento del árbol, es importante tener en cuenta el rol que cumplen los datos del dataset a la hora de entrenar al modelo, y luego testarlo. Dado que utilizaremos el método *k-folding* para elegir al árbol con mejor performance, debemos dividir cuidadosamente los datos antes de usarlos para entrenar al modelo.

Inicialmente dividimos los datos en dos grupos principales, datos de *desarrollo* y datos *held-out*. *Desarrollo* más tarde será dividido en los grupos que ya usamos anteriormente, *train* y *test*, utilizados para entrenar al modelo y testarlo, respectivamente. Mientras que *held-out* será reservado exclusivamente para cuando el árbol ya haya sido entrenado y testeado. Proporcionando así una manera de evaluarlo finalmente con datos que desconoce por completo, dando una noción más precisa de su performance. La proporción en la que dividimos los datos fue de 90% *desarrollo* (72% *train*, 18% *test*) y 10% *held-out*.

Comenzamos analizando la variación de exactitud de los árboles con distintas profundidades. En este punto, los árboles fueron entrenados usando el conjunto *Train*, y la exactitud fue evaluada sobre las predicciones del conjunto *Test*. Ajustamos un modelo para cada profundidad del 1 al 10.

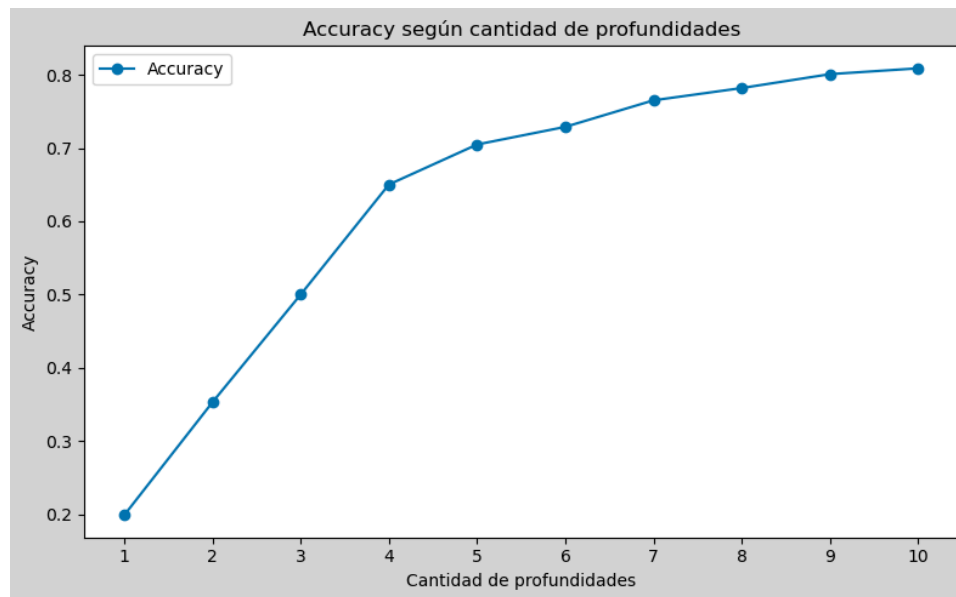


Fig 11: Accuracy de los árboles en función de su cantidad de profundidades

Observamos en la Fig. 11 que el rendimiento aumenta junto con las profundidades, siendo el árbol con 10 niveles el que mejor exactitud alcanzó. A su vez, si analizamos cómo varía la exactitud del modelo con la profundidad del árbol, no hay señales claras de sobreajuste dentro de las profundidades consideradas. Este factor es muy importante a la hora de ajustar este tipo de modelos, sobre o sub ajustarlos podría perjudicarnos.

Para obtener una medida de performance más refinada, a partir de ahora aplicaremos *k-folding*. Este método se basa en dividir en k grupos nuestros datos de *desarrollo*, e ir alternándolos para entrenar y testear al modelo múltiples veces. Cada vez se va guardando la exactitud, y luego de terminar el proceso de validación cruzada, se promedian las exactitudes obtenidas, generando una que refleja de manera más refinada la verdadera performance del modelo. Utilizamos la medida general $k = 5$, dado que es apropiado para la cantidad de datos que tenemos; al trabajar con bases de datos más grandes se suele utilizar $k = 10$ pero para nuestro caso no lo consideramos óptimo.

Luego, nos planteamos encontrar la combinación de hiperparámetros ideal para lograr la mejor performance posible con un árbol de decisión. Para abordarlo, diseñamos una función que compara la exactitud usando validación cruzada (*k-folding*) de todos los posibles árboles de decisión que puedan crearse a partir de los hiperparámetros ingresados. Luego de seleccionar una cantidad abarcativa de los hiperparámetros útiles, obtuvimos la mejor configuración de hiperparámetros posibles dentro de los ingresados. La exactitud de este ronda en 80.9%, y los hiperparámetros utilizados para ajustarlo fueron como criterio entropía y 10 niveles de profundidad.

Tras escoger el mejor árbol, posterior a aplicar *k-folding*, lo evaluamos utilizando el conjunto de datos que habíamos dejado sellado, el *held-out*. Para medir su performance, escogimos exactitud, dado que en nuestro caso estamos trabajando sobre una base de datos balanceada y no es crítico confundir una clase al clasificar una imagen. Si no fuera balanceada, como la exactitud calcula la proporción de predicciones correctas sobre todas las muestras, entonces podríamos estar subestimando la performance en las clases con menos datos. Además, al no estar buscando priorizar minimizar falsos negativos o falsos



positivos, dado que no es grave confundir una clase con otra en nuestro caso, luego concluimos que exactitud es una métrica óptima para utilizar. Esta fue de 81.3%.

A su vez, decidimos armar la matriz de confusión del mejor árbol, la cuál es una herramienta utilizada para evaluar el desempeño de modelos de clasificación. Permite comparar las predicciones del modelo con los datos reales, organizando los aciertos y errores en una tabla. Es muy útil para identificar qué clases están correctamente clasificadas y cuáles tienden a confundirse, y así podemos analizar más profundamente por qué se equivoca más en ciertas clases que en las demás. Así podrían mejorarse los modelos. Nuestra matriz de confusión, como era de esperarse, reflejó que hay ciertas clases, las más parecidas entre ellas, donde el modelo se equivoca con mayor frecuencia. Tiene sentido dado que, como verificamos en el análisis exploratorio, son clases muy parecidas, lo que da lugar a que el modelo las confunda y se equivoque. A continuación ilustramos la matriz de confusión obtenida en este caso:

Matriz de confusión

	0	1	2	3	4	5	6	7	8	9
0	573	1	12	37	2	0	69	0	6	0
1	10	655	4	27	1	0	2	0	1	0
2	8	3	452	3	193	0	37	0	4	0
3	38	13	12	568	51	1	17	0	0	0
4	4	0	60	24	574	0	34	0	4	0
5	0	2	0	1	3	612	2	54	5	21
6	131	1	116	24	114	1	302	0	11	0
7	0	0	0	0	0	13	1	647	1	38
8	10	2	14	6	14	9	13	5	626	1
9	1	1	1	0	2	7	1	58	3	626
	0	1	2	3	4	5	6	7	8	9

Clase verdadera

Clase predicha

Fig. 12: Matriz de confusión del árbol de decisión con mejor performance.

Podemos observar en la Fig. 12 como el modelo predice en general de manera acertada, pero falla considerablemente en algunas clases. Es fácil ver que falla en mayor proporción en la clase 6, confundiendola con las clases 0, 2 y 4. Es lógico dado que estamos comparando camisas con remeras, pullovers y camperas. Podríamos intuir que aquellas remeras de manga larga fueron confundidas con el resto de las clases mencionadas, dado que su forma es muy parecida (como se ve en la Fig. 2). De nuevo, es fácil ver que muchos pullovers (clase 2) fueron mal clasificados como camperas. A su vez, podemos ver reflejado en la matriz de confusión el hecho de que los pantalones (clase 1) tienen una gran similitud interna en su clase, pues son la clase que mejor performance obtuvo. Es claro que el modelo logra diferenciarla bien del resto de clases.

Teniendo en cuenta las circunstancias (varias clases muy parecidas entre sí), podemos concluir que este modelo logra un rendimiento satisfactorio a la hora de clasificar

una imagen entre muchas posibles clases. Vemos entonces que los árboles de decisión son una buena elección para tratar la clasificación multiclase.

Conclusión

Luego de evaluar y comparar modelos de clasificación binaria basados en KNN y clasificación multiclase basados en el método árboles de decisión, logramos obtener modelos que logran predecir, con una exactitud relativamente alta, a qué clase pertenece una imagen.

Para clasificación binaria, encontramos que el criterio con el cual seleccionamos que píxeles a tener en cuenta para hacer KNN fue muy importante. Reconocemos además que no fue trivial que nuestro objetivo sea diferenciar entre clases que se distinguen bastante. El mejor modelo que encontramos tuvo una exactitud de 94.46%, el cual trabajaba con un conjunto de diez píxeles obtenidos mediante el tercer criterio que desarrollamos, y con un valor de $K=6$.

Para clasificación multiclase, observamos que el árbol de decisión resultó ser un modelo satisfactorio para predecir entre múltiples clases posibles. Aquel que mejor desempeño tuvo fue ajustado con 10 niveles de profundidad y criterio de entropía. Por más que su exactitud luego de validar con el conjunto *held-out* fue de 81.3%, lo cual pareciera ser desalentador, pues es menor que en clasificación binaria, resaltamos los factores que inciden en este resultado. Por un lado, las clases altamente diferenciables gozaron de un rendimiento adecuado. Mientras que por otro lado, observamos el obstáculo que tendría cualquier modelo a la hora de tener que tratar con clases muy similares; esta dificultad la observamos plasmada en la matriz de confusión, donde queda en evidencia el rendimiento entre las clases con mayor similitud visual. Esta performance más baja, acaba reduciendo la exactitud general del modelo.

Recalcamos la importancia de elaborar y analizar una matriz de confusión, para así comprender mejor el comportamiento del modelo clasificador. Así como también, el rol fundamental que cumple el análisis previo a seleccionar la mejor configuración de hiperparámetros para ajustar un modelo que permita obtener un rendimiento óptimo.